KẾ HOẠCH NÂNG CẤP REELS VÀ HỆ THỐNG GỢI Ý LÊN MỨC SẢN PHẨM

Dự án Mobile Social Network (Expo/React Native, NestJS, Prisma, Supabase)
Ngày lập: 12/10/2026. Tài liệu đề xuất, chưa thay đổi mã nguồn, schema hay API.

---

1. TÓM TẮT

Tài liệu này nối tiếp `docs/reel/REEL_FINAL_SPEC_AND_PLAN.md`. Bản đó là kế hoạch MVP trong khoảng 5 tuần và chủ động loại bỏ chuyển mã video, embedding và học máy. Mục tiêu ở đây khác: đưa tính năng lên môi trường thật và làm hệ thống gợi ý giống các nền tảng lớn nhất có thể, trong khi vẫn dùng lại tối đa những gì dự án đang có.

Bốn kết luận chính:

1. **"Giống hệ thống lớn" gồm hai phần.** Phần kiến trúc (phễu nhiều tầng, nhiều nguồn ứng viên, log hành vi, thử nghiệm A/B, phương án dự phòng) làm được ngay từ đầu. Phần mô hình học máy chỉ tốt khi có dữ liệu: với vài trăm người dùng, một mô hình sâu thường thua công thức chấm điểm được chỉnh cẩn thận.
2. **Có bảy phương án, khuyến nghị Phương án 7.** Giữ engine của bản MVP làm nền và dự phòng (Phương án 1), thêm truy hồi bằng embedding trên pgvector (Phương án 2), rồi thay tầng xếp hạng bằng mô hình học từ log khi đủ dữ liệu (Phương án 5). Kiến trúc đích chính là kiến trúc của hệ thống lớn.
3. **Phần tốn công nhất để deploy không phải thuật toán gợi ý.** Đó là đường ống video (nén, HLS, CDN), kiểm duyệt nội dung và log sự kiện. Ba việc này cần cho mọi phương án.
4. **Ước lượng cho nhóm hai người:** bản MVP 5 tuần như đã lên kế hoạch, thêm khoảng 2 tuần để sẵn sàng production, 2 tuần cho embedding, và 5 đến 8 tuần cho các tầng học máy. Phần học máy chỉ nên bắt đầu khi đã có log thật.

2. HIỆN TRẠNG DỰ ÁN VÀ KHOẢNG TRỐNG

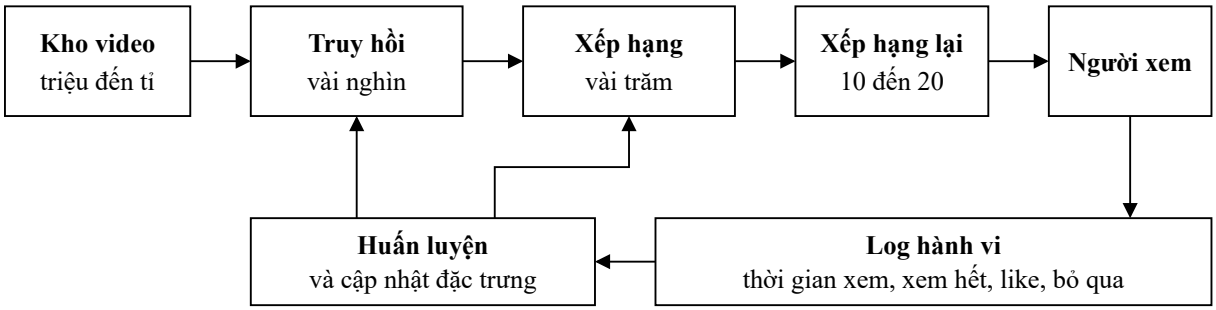
Bảng dưới đây lập từ mã nguồn hiện tại (nhánh `main`, commit `fc04616`). Chưa có phần nào của bản MVP Reels được cài đặt.

Hạng mục	Đã có	Còn thiếu
Nội dung	`Post`, `PostMedia` (cột `type` kiểu String), `PostPrivacy` (PUBLIC, FRIENDS, PRIVATE), `Repost`	Thông tin video (thời lượng, kích thước, thumbnail, trạng thái xử lý), cờ `reelEligible`. Bản MVP viết trước khi có quyền riêng tư và repost nên chưa tính hai thứ này.

Hạng mục	Đã có	Còn thiếu
Tương tác	`Like`, `Comment` (có trả lời), `Follow`, `Notification` kèm Socket.IO	Log lượt xem và thời gian xem, lưu bài, "không quan tâm", chặn và báo cáo.
Upload	`StorageService.upload` nhận buffer từ multer rồi đẩy lên Supabase Storage, chỉ nhận ảnh	Video không thể đi qua RAM của API. Chưa có signed upload URL, chưa nén hay chuyển mã.
Feed	`PostsService.getFeed`: sắp theo thời gian, phân trang page/limit, lọc quyền riêng tư, trộn repost	Chưa có xếp hạng, chưa lọc nội dung đã xem.
Mobile	Expo SDK 57, `FlatList`, `expo-image`, `expo-image-picker`, Socket.IO client	`expo-video`, màn hình Reels, bộ đo thời gian xem.
Hạ tầng	Supabase Postgres qua transaction pooler, một tiến trình NestJS	Redis, hàng đợi job, tác vụ định kỳ, pgvector, dịch vụ ML, giám sát.

3. CÁC HỆ THỐNG LỚN HOẠT ĐỘNG THẾ NÀO

YouTube, TikTok, Instagram Reels và Kuaishou đều dùng cùng một khung: một phễu nhiều tầng thu hẹp dần số video, và một vòng phản hồi biến hành vi người xem thành dữ liệu huấn luyện.



Hình 1. Phễu gợi ý nhiều tầng và vòng phản hồi

- **Truy hồi (retrieval)** chạy nhiều kênh song song: mô hình two-tower với tìm kiếm láng giềng gần đúng, lọc cộng tác theo video ("người xem X cũng xem Y"), đồ thị follow, xu hướng, video mới. Tầng này ưu tiên không bỏ sót và phải rất nhanh.
- **Xếp hạng (ranking)** dùng một mô hình đa nhiệm dự đoán cùng lúc xác suất xem hết, like, bình luận, chia sẻ, bỏ qua và thời gian xem kỳ vọng. Điểm cuối là tổ hợp có trọng số của các dự đoán đó. Bộ trọng số là quyết định sản phẩm và được chỉnh bằng thử nghiệm A/B.
- **Xếp hạng lại (re-ranking)** áp các luật: không lặp tác giả, đa dạng chủ đề, chèn video mới để thăm dò, lọc an toàn.

- **Tín hiệu ngầm là chính.** Thời gian xem, tỉ lệ xem hết, xem lại và lướt qua nhanh quan trọng hơn like, vì mỗi lần vuốt đều sinh ra dữ liệu.
- **Khởi động nguội (cold start).** Video mới được cấp một lượng hiển thị nhỏ. Nếu tỉ lệ xem hết vượt ngưỡng, nó được đưa lên nhóm lưu lượng lớn hơn.
- **Thời gian thực có hai mức.** Cập nhật đặc trưng theo thời gian thực (sở thích trong phiên đổi ngay sau mỗi lượt vuốt) rẻ và nên làm trước. Huấn luyện mô hình theo thời gian thực (như Monolith của ByteDance) đắt hơn nhiều bậc và chỉ có ý nghĩa ở lưu lượng rất lớn.

4. NỀN CHUNG CHO MỌI PHƯƠNG ÁN

Các mục dưới đây không phụ thuộc vào việc chọn thuật toán gợi ý nào. Thiếu một trong số chúng thì không deploy được, hoặc deploy được nhưng không cải thiện được về sau.

4.1. Đường ống video

Lựa chọn	Ưu điểm	Nhược điểm
V1. MP4 nén trên máy người dùng, tải thẳng lên Supabase Storage bằng signed URL (như bản MVP)	Không thêm dịch vụ. Dùng lại `StorageService`. Làm nhanh nhất.	Chỉ một mức chất lượng nên mạng yếu sẽ giật. Băng thông Supabase tính phí khi vượt gói. Chất lượng nén phụ thuộc thiết bị.
V2. Tự chuyển mã HLS: hàng đợi job, worker ffmpeg, lưu trên R2 hoặc S3, phát qua CDN	Nhiều mức bitrate, tự đổi theo mạng. Toàn quyền kiểm soát. Chi phí lưu trữ và băng thông thấp nhất về lâu dài.	Phải vận hành worker tốn CPU, hàng đợi và xử lý lỗi. Thêm 1 đến 2 tuần.
V3. Dịch vụ video quản lý (Cloudflare Stream, Mux, Bunny Stream)	Có HLS, thumbnail và CDN trong vài ngày. Tải lên trực tiếp, báo xong qua webhook.	Phí theo phút lưu trữ và phút xem. Phụ thuộc nhà cung cấp.

Khuyến nghị: V1 cho bản nội bộ, V3 trước khi mở cho người dùng thật, và chuyển sang V2 khi phí của V3 trở nên đáng kể. Trường `storageKey` và `status` trong thiết kế MVP đã cho phép đổi nhà cung cấp mà không đổi schema.

4.2. Trình phát trên mobile

Giữ thiết kế ở mục 7 của bản MVP (`FlatList` với `pagingEnabled`, `expo-video`, chỉ video đang hiển thị được phát). Để đạt mức sản phẩm cần thêm:

- Chỉ giữ ba trình phát trong bộ nhớ (trước, hiện tại, sau) và nạp trước video kế tiếp.
- Dừng phát khi rời tab hoặc khi ứng dụng xuống nền.

- Đo thời gian tới khung hình đầu tiên và tỉ lệ lỗi phát, gửi về cùng lô sự kiện.
- Lưu sự kiện chưa gửi và gửi lại khi có mạng.

4.3. Log sự kiện

Đây là tài sản quan trọng nhất của hệ thống gợi ý, vì mọi mô hình về sau đều học từ nó. Bản MVP đã có `ContentInteraction` và `RecommendationLog`. Cần bổ sung ngay từ đầu, vì dữ liệu đã ghi thiếu thì không bù lại được:

- `requestId` và `position`: nối mỗi lượt xem với lô gợi ý đã trả về. Không có hai cột này thì không huấn luyện được mô hình xếp hạng và không khử được thiên lệch vị trí.
- `source` và `modelVersion`: biết video đến từ kênh nào và phiên bản nào, để đánh giá từng kênh và làm A/B.
- `eventId` do client sinh: gửi lại không tạo bản ghi trùng.
- Ghi cả lượt hiển thị không có tương tác. Mô hình cần ví dụ âm.

4.4. An toàn nội dung và quyền riêng tư

- **Quyền riêng tư bài viết.** Nguồn gợi ý chỉ lấy bài `PUBLIC`. Bài `PRIVATE` không bao giờ xuất hiện. Bài `FRIENDS` hoặc bị loại hẳn khỏi Reels, hoặc chỉ hiện cho người follow hai chiều theo đúng điều kiện `getFeed` đang dùng. Bộ lọc này phải chạy lại ngay trước khi trả kết quả.
- **Chặn và báo cáo.** Thêm `Block` và `Report`. Apple App Store và Google Play đều yêu cầu ứng dụng có nội dung do người dùng tạo phải có cơ chế báo cáo, chặn và gỡ nội dung.
- **Kiểm duyệt trước khi phân phối.** Video chỉ vào nguồn gợi ý sau khi qua kiểm duyệt (tự động hoặc thủ công). Video chưa duyệt vẫn hiện trên trang cá nhân của tác giả.
- **Dữ liệu hành vi.** Chính sách quyền riêng tư phải nêu việc thu thập lịch sử xem. Người dùng cần xóa được lịch sử. Cần đối chiếu quy định bảo vệ dữ liệu cá nhân của Việt Nam trước khi phát hành.

4.5. Hạ tầng tối thiểu

- **Redis:** cache nhóm ứng viên, danh sách đã xem, đặc trưng trong phiên, hàng đợi job.
- **Tác vụ định kỳ:** `pg_cron` có sẵn trên Supabase hoặc `@nestjs/schedule`, dùng để tổng hợp `PostStats` theo lô mỗi vài phút thay vì ghi trên từng sự kiện.
- **Giám sát:** độ trễ p95 của endpoint feed, tỉ lệ lỗi, tỉ lệ rơi về phương án dự phòng.
- **Giới hạn tần suất** cho endpoint nhận sự kiện, để chống bơm lượt xem.

4.6. Thay đổi schema bổ sung so với bản MVP

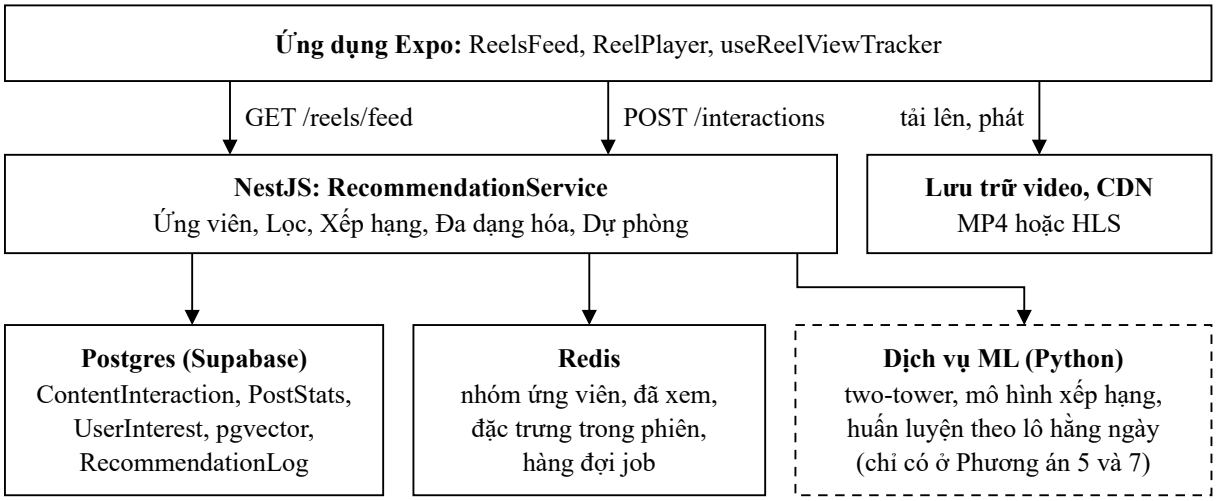
```
model ContentInteraction {      // giữ các cột của bản MVP, thêm:
  eventId      String  @unique  // client sinh, chống ghi trùng
  requestId    String?          // nối với RecommendationLog
  position     Int?             // vị trí trong lô được trả về
  source       String?          // kênh ứng viên
  modelVersion String?
}
model PostMedia {               // thêm:
  hlsUrl     String?
  moderation String @default("PENDING")   // PENDING | APPROVED | REJECTED
}
model Block  { blockerId String; blockedId String; createdAt DateTime
               @@id([blockerId, blockedId]) }
model Report { id String @id; reporterId String; postId String
               reason String; status String; createdAt DateTime }

// Chỉ cần từ Phương án 2 trở đi:
model PostEmbedding { postId String @id; model String
                      embedding Unsupported("vector(512)") }
model UserVector    { userId String; slot Int; updatedAt DateTime
                      embedding Unsupported("vector(512)")
                      @@id([userId, slot]) }
```

Tất cả là đề xuất. Theo RULE.md điều 8 và 9, schema và API phải được Dev B duyệt và ghi vào `docs/api-contract.md` trước khi cài đặt.

5. CÁC PHƯƠNG ÁN CHO HỆ THỐNG GỢI Ý

Cả bảy phương án đều cắm vào cùng một chỗ: `RecommendationService` trong `apps/api`, phía sau `GET /reels/feed` và `GET /posts/for-you` đã đề xuất ở bản MVP. Ứng dụng mobile không cần biết phương án nào đang chạy.



Hình 2. Kiến trúc đích trong dự án hiện tại

Phương án 1. Chấm điểm theo luật trên Postgres

Ý tưởng. Đây là engine của bản MVP: ứng viên từ năm nguồn (follow, chủ đề, xu hướng, video mới, tác giả quen), lọc, chấm điểm `0,30·chủ đề + 0,25·chất lượng + 0,25·quan hệ + 0,20·độ mới`, rồi đa dạng hóa. Sở thích người dùng lưu theo chủ đề trong `UserInterest`.

Nâng cấp cho production. Thêm bộ lọc quyền riêng tư, chặn và kiểm duyệt. Tổng hợp `PostStats` theo lô. Cấp lưu lượng theo nhóm cho video mới (ví dụ 200, 1.000, 5.000 lượt hiển thị, lên nhóm khi tỉ lệ xem hết vượt ngưỡng). Cache nhóm xu hướng và video mới trong Redis. Gắn bộ trọng số với nhóm A/B.

Tích hợp. Toàn bộ nằm trong `apps/api` bằng TypeScript và Prisma. Thứ duy nhất phải thêm là Redis.

Ưu điểm.

- Nhanh nhất và đã có kế hoạch chi tiết.
- Chạy tốt khi chưa có dữ liệu, độ trễ thấp.
- Giải thích được từng kết quả ("vì sao tôi thấy video này").
- Dùng làm phương án dự phòng cho mọi phương án còn lại.

Nhược điểm.

- Phụ thuộc hashtag và từ khóa. Video không có chú thích thì hệ thống không hiểu.
- Không học được quan hệ ngầm giữa các video hay giữa những người xem giống nhau.
- Trọng số chỉnh tay, trần chất lượng thấp.
- Giống hệ thống lớn ở kiến trúc nhưng không giống ở mô hình.

Công sức. Nằm trong 5 tuần của bản MVP, cộng khoảng 1 tuần cho phần nâng cấp.

Phương án 2. Truy hồi bằng embedding trên pgvector

Ý tưởng. Mỗi video có một vector nội dung tính từ chú thích và vài khung hình (có thể thêm lời thoại). Mỗi người dùng có 3 đến 5 vector sở thích, là trung bình có trọng số theo thời gian của các video họ xem hết hoặc like, tách thành cụm để các sở thích khác nhau không bị trộn lẫn (cách của PinnerSage). Ứng viên là các video gần những vector đó nhất. Tầng xếp hạng vẫn là công thức của Phương án 1, thêm thành phần độ tương đồng.

Tích hợp. Bật extension `vector` trên Supabase. Trong Prisma khai báo cột bằng `Unsupported("vector(n)")` và truy vấn qua `$queryRaw`, dùng chỉ mục HNSW. Một worker tính embedding sau khi video tải lên xong, bằng mô hình mã nguồn mở họ CLIP hoặc SigLIP chạy trên CPU, hoặc bằng API embedding đa phương thức. Vector người dùng được cập nhật ngay trong xử lý `POST /interactions`, nên gợi ý đổi trong chính phiên xem.

Ưu điểm.

- Giải quyết khởi động nguội cho video: có vector ngay khi tải lên, chưa cần lượt xem nào.
- Không phụ thuộc hashtag.
- Nằm ngay trong database đang dùng, không thêm hệ quản trị mới.
- Vector dùng lại được cho tìm kiếm, phát hiện video trùng và làm đặc trưng cho Phương án 5.

Nhược điểm.

- Giống nhau về nội dung không có nghĩa là hay. Vẫn cần tín hiệu chất lượng từ Phương án 1.
- Dễ tạo buồng vọng nếu không chèn nội dung khám phá.
- Cần worker tính embedding, tốn CPU hoặc phí API theo số video.
- Truy vấn vector kèm bộ lọc (đã xem, quyền riêng tư) cần chỉnh cẩn thận để không trả thiếu kết quả.
- Đổi mô hình embedding thì phải tính lại toàn bộ.

Công sức. Khoảng 2 tuần sau khi có Phương án 1.

Phương án 3. Gorse, engine gợi ý mã nguồn mở tự vận hành

Ý tưởng. Gorse nhận người dùng, nội dung và phản hồi qua REST API, tự huấn luyện lọc cộng tác, gợi ý nội dung tương tự và mô hình xếp hạng, rồi trả về danh sách gợi ý. Có sẵn bảng điều khiển.

Tích hợp. Chạy bằng Docker cạnh API, cần Redis và một database. NestJS đẩy nội dung khi tạo bài, đẩy phản hồi trong `POST /interactions`, và gọi Gorse như một kênh ứng viên trong `RecommendationService`. Lọc quyền riêng tư và đa dạng hóa vẫn làm ở NestJS.

Ưu điểm.

- Có lọc cộng tác thật trong 1 đến 2 tuần mà không phải viết mã học máy.
- Tự huấn luyện lại, có bảng điều khiển theo dõi.
- Mã nguồn mở, dữ liệu ở lại máy chủ của nhóm.

Nhược điểm.

- Phản hồi chủ yếu ở dạng có hoặc không, khó tận dụng thời gian xem, vốn là tín hiệu chính của video ngắn.
- Ít quyền kiểm soát mô hình và mục tiêu tối ưu. Không hiểu nội dung video.
- Thêm một hệ thống phải vận hành và một máy chủ chạy Docker.
- Khi cần vượt qua giới hạn của nó thì phải xây lại theo Phương án 5.

Công sức. 1 đến 2 tuần, cộng chi phí một máy chủ nhỏ.

Phương án 4. Dịch vụ gợi ý quản lý

Ý tưởng. Gửi danh mục nội dung và sự kiện cho nhà cung cấp như AWS Personalize, Recombee hoặc Shaped. Họ huấn luyện và phục vụ mô hình, API trả về danh sách ID đã xếp hạng.

Tích hợp. NestJS chuyển tiếp sự kiện từ `POST /interactions` và gọi API gợi ý trong `RecommendationService`. Vẫn phải tự lọc quyền riêng tư và giữ Phương án 1 làm dự phòng khi dịch vụ lỗi hoặc chậm.

Ưu điểm.

- Mô hình chất lượng cao mà không cần người chuyên về học máy.
- Có sẵn xử lý khởi động nguội, cập nhật theo sự kiện và công cụ thử nghiệm.
- Không phải lo mở rộng.

Nhược điểm.

- Chi phí định kỳ tăng theo lưu lượng và khó dự đoán.
- Hộp đen: khó gỡ lỗi, khó giải thích kết quả.
- Khóa vào nhà cung cấp. Dữ liệu hành vi người dùng rời khỏi hệ thống, cần xem xét quy định chuyển dữ liệu ra nước ngoài.
- Thêm độ trễ mạng cho mỗi lần lấy feed.
- Không đáp ứng mục tiêu tự làm chủ một hệ thống giống các nền tảng lớn.

Công sức. 1 đến 2 tuần. Chi phí theo bảng giá nhà cung cấp, cần kiểm tra tại thời điểm chọn.

Phương án 5. Tự xây hệ thống nhiều tầng bằng học máy

Ý tưởng. Bản thu nhỏ của kiến trúc YouTube, TikTok và Instagram, gồm các thành phần sau.

Thành phần	Nội dung
Kho đặc trưng	Redis cho phục vụ trực tuyến, bảng Postgres hoặc tệp Parquet cho huấn luyện. Cùng một đoạn mã tính đặc trưng cho cả hai phía.
Truy hồi đa kênh	Two-tower (tháp người dùng: ID và chuỗi video xem gần nhất; tháp video: embedding nội dung, ID, tác giả), huấn luyện bằng in-batch softmax có hiệu chỉnh tần suất. Chỉ mục pgvector HNSW, đổi sang Qdrant hoặc FAISS khi lớn. Thêm các kênh lọc cộng tác theo video, follow, xu hướng, video mới.
Xếp hạng đa mục tiêu	Bắt đầu bằng LightGBM, mỗi mục tiêu một mô hình (xem hết, like, chia sẻ, bỏ qua, thời gian xem). Sau đó chuyển sang MMoE hoặc PLE trên PyTorch. Hiệu chỉnh thiên lệch do thời lượng video và do vị trí.
Xếp hạng lại	MMR hoặc DPP để đa dạng hóa, luật về tác giả và chủ đề, chèn nội dung khám phá.

Thành phần	Nội dung
Phục vụ	Dịch vụ Python (FastAPI) riêng. NestJS gọi với thời hạn khoảng 150 ms, quá hạn thì rơi về Phương án 1.
Huấn luyện	Theo lô hằng ngày từ log. Mô hình mới chạy ở chế độ bóng (chỉ ghi điểm, không ảnh hưởng người dùng), sau đó A/B. Lưu phiên bản mô hình.

Tích hợp. Một thư mục mới trong repo (ví dụ `services/recsys/`), dùng chung Postgres và Redis. Trong `apps/api` chỉ `RecommendationService` thay đổi.

Ưu điểm.

- Giống hệ thống lớn nhất trong các phương án khả thi với nhóm nhỏ.
- Trần chất lượng cao nhất, toàn quyền về mục tiêu tối ưu và dữ liệu.
- Thêm kênh hoặc đổi mô hình mà không đổi kiến trúc.
- Kiến thức và mã nguồn thuộc về nhóm.

Nhược điểm.

- Cần dữ liệu. Theo kinh nghiệm chung, dưới vài trăm nghìn lượt hiển thị có log thì mô hình học chưa hơn được công thức của Phương án 1.
- Công sức lớn nhất. Thêm ngôn ngữ thứ hai và một pipeline huấn luyện phải vận hành.
- Nhiều kiểu lỗi khó thấy: đặc trưng lúc huấn luyện khác lúc phục vụ, rò rỉ nhãn, vòng phản hồi tự khuếch đại.
- Phải giám sát chất lượng mô hình liên tục.

Công sức. 5 đến 8 tuần sau khi đã có Phương án 1 và log. Một máy chủ CPU 2 đến 4 nhân đủ cho cả phục vụ và huấn luyện ở quy mô đầu, chưa cần GPU.

Phương án 6. Luồng sự kiện thời gian thực và học trực tuyến

Ý tưởng. Sự kiện đi qua Kafka hoặc Redpanda. Một bộ xử lý luồng như Flink tính đặc trưng và sinh mẫu huấn luyện liên tục. Mô hình cập nhật tham số mỗi vài phút, theo hướng của Monolith.

Tích hợp. Luồng sự kiện thay bảng `ContentInteraction` làm nguồn chính, Postgres chỉ nhận số liệu tổng hợp. Cần cụm Kafka, bộ xử lý luồng và máy chủ tham số.

Ưu điểm.

- Độ tươi của mô hình cao nhất. Đây là thứ tạo khác biệt của TikTok.
- Chịu được hàng trăm triệu sự kiện mỗi ngày.

Nhược điểm.

- Chi phí và khối lượng vận hành vượt xa một nhóm hai người.
- Học trực tuyến rất khó gỡ lỗi và dễ hỏng âm thầm.

- Ở lưu lượng nhỏ, phần lớn lợi ích đạt được rẻ hơn nhiều bằng cách cập nhật đặc trưng trong Redis.

Kết luận. Chưa làm. Chỉ cần thiết kế để `POST /interactions` có thể đổi đích ghi sang hàng đợi mà không đổi phía mobile.

Phương án 7. Kết hợp theo giai đoạn (khuyến nghị)

Ý tưởng. Không chọn một phương án mà xếp chồng chúng. Phương án 1 là kênh nền và dự phòng. Phương án 2 thêm kênh truy hồi theo nội dung và cập nhật sở thích trong phiên. Phương án 5 thay tầng xếp hạng trước, rồi thêm two-tower sau. Mỗi bước chỉ được bật cho toàn bộ người dùng khi thắng A/B so với bước trước.

Ưu điểm.

- Lúc nào cũng có một bản chạy được để deploy.
- Mỗi bước dùng lại trọn vẹn bước trước: log của Phương án 1 là dữ liệu huấn luyện của Phương án 5, vector của Phương án 2 là đặc trưng của Phương án 5.
- Kiến trúc cuối cùng chính là Phương án 5, tức là giống hệ thống lớn.
- Rủi ro được chia nhỏ, có thể dừng ở bất kỳ giai đoạn nào.

Nhược điểm.

- Tổng thời gian dài nhất.
- Phải giữ kỷ luật log và A/B ngay từ đầu.
- Trong vài tháng đầu, hệ thống đơn giản hơn kỳ vọng "giống TikTok".

6. SO SÁNH TỔNG HỢP

Phương án	**Giống hệ thống lớn**	**Chất lượng khi ít dữ liệu**	**Trần chất lượng**	**Công sức (2 người)**	**Vận hành**	**Tích hợp vào dự án**
1. Luật trên Postgres	Thấp	Tốt	Thấp	MVP + 1 tuần	Dễ	Rất tốt
2. Embedding, pgvector	Trung bình	Tốt	Trung bình	+ 2 tuần	Dễ	Rất tốt
3. Gorse	Trung bình	Trung bình	Trung bình	1 đến 2 tuần	Trung bình	Tốt
4. Dịch vụ quản lý	Trung bình	Khá	Cao	1 đến 2 tuần	Dễ	Tốt
5. Tự xây bằng ML	Cao	Kém nếu đứng một mình	Cao	5 đến 8 tuần	Khó	Khá
6. Thời gian thực	Rất cao	Kém	Rất cao	Nhiều tháng	Rất khó	Kém
7. Kết hợp	**Cao**	**Tốt**	**Cao**	**9 đến 12 tuần sau MVP**	**Tăng dần**	**Rất tốt**

Các mức trong bảng là đánh giá định tính của người lập. Công sức là ước lượng, chưa tính thời gian chờ tích lũy dữ liệu.

7. LỘ TRÌNH CHO PHƯƠNG ÁN 7

Giai đoạn	**Backend**	**Mobile**	**Điều kiện hoàn thành**
GĐ0. Bản MVP (khoảng 5 tuần)	Phase 0 đến 6 của bản MVP. Bổ sung ngay năm cột log ở mục 4.6 và bộ lọc quyền riêng tư.	Theo bản MVP. Tracker gửi thêm `eventId`, `requestId`, `position`.	Nghiệm thu Phase 5 của bản MVP: hai tài khoản có hành vi khác nhau nhận feed khác nhau.
GĐ1. Sẵn sàng production (khoảng 2 tuần)	Video HLS (lựa chọn V3). `Block`, `Report`, trạng thái kiểm duyệt. Redis. `PostStats` theo lô. Nhóm lưu lượng cho video mới. Phân nhóm A/B bằng băm `userId`. Giám sát.	Phát HLS, nạp trước. Màn hình báo cáo và chặn. Đo thời gian tới khung hình đầu. Gửi lại sự kiện khi mất mạng.	Độ trễ p95 của feed dưới 300 ms. Video bắt đầu phát dưới 1 giây trên 4G. Qua thử tải.

Giai đoạn	Backend	Mobile	Điều kiện hoàn thành
GĐ2. Embedding (khoảng 2 tuần)	pgvector, worker embedding, `UserVector`, kênh ứng viên theo độ tương đồng, cập nhật vector trong phiên.	Không đổi API. Tùy chọn: mục "video tương tự".	Thắng Phương án 1 trong A/B về thời gian xem mỗi phiên. Video không có hashtag vẫn tới đúng người.
GĐ3. Xếp hạng bằng ML (3 đến 4 tuần)	Dịch vụ Python, kho đặc trưng, LightGBM đa mục tiêu, chế độ bóng rồi A/B, dự phòng khi quá hạn.	Không đổi.	Chỉ bắt đầu khi có vài trăm nghìn lượt hiển thị. AUC ngoại tuyến vượt baseline. A/B không làm giảm tỉ lệ giữ chân.
GĐ4. Truy hồi bằng ML (2 đến 4 tuần)	Two-tower, lọc cộng tác theo video, MMoE, đa dạng hóa bằng DPP.	Không đổi.	Recall@K ngoại tuyến tăng và thắng A/B.
GĐ5. Theo lưu lượng	Hàng đợi sự kiện, tách kho log khỏi Postgres, cân nhắc Phương án 6.	Không đổi.	Chỉ làm khi Postgres không còn chịu được lượng sự kiện.

Trong lúc chưa có người dùng thật: dùng các bộ dữ liệu công khai về video ngắn như KuaiRec, KuaiRand hoặc MicroLens để dựng và kiểm tra pipeline huấn luyện của GĐ3 và GĐ4. Khi có log thật chỉ cần đổi nguồn dữ liệu.

8. ĐO LƯỜNG VÀ THỬ NGHIỆM

- **Chỉ số trực tuyến:** thời gian xem mỗi phiên, tỉ lệ xem hết, tỉ lệ bỏ qua dưới 2 giây, số video mỗi phiên, tỉ lệ like, chia sẻ và follow trên lượt hiển thị, tỉ lệ giữ chân sau 1 và 7 ngày, tỉ lệ "không quan tâm" và báo cáo.
- **Sức khỏe hệ sinh thái:** tỉ lệ video mới nhận đủ lượt hiển thị đầu tiên, độ phủ danh mục, mức tập trung lượt xem vào nhóm video đứng đầu, độ đa dạng chủ đề trong một phiên.
- **Chỉ số ngoại tuyến:** Recall@K cho truy hồi, AUC và NDCG cho xếp hạng. Chia tập huấn luyện và kiểm tra theo thời gian, không chia ngẫu nhiên.
- **A/B:** phân nhóm bằng băm `userId` cùng tên thử nghiệm, ghi `modelVersion` vào log, mỗi lần chỉ đổi một thứ, và giữ lâu dài một nhóm đối chứng nhỏ chạy Phương án 1.
- **Chỉ số kỹ thuật:** độ trễ p95 của feed, tỉ lệ rơi về dự phòng, thời gian tới khung hình đầu, tỉ lệ lỗi phát.

9. RỦI RO VÀ CÁCH GIẢM THIỂU

Rủi ro	Giảm thiểu
Không đủ dữ liệu cho học máy	Đi theo Phương án 7. Dựng pipeline trên dữ liệu công khai. Đặt ngưỡng dữ liệu trước GĐ3.
Chi phí lưu trữ và băng thông video	Giới hạn 60 giây. HLS nhiều mức. CDN. Theo dõi chi phí trên mỗi nghìn lượt xem.
Nội dung vi phạm lọt vào feed gợi ý	Kiểm duyệt trước khi vào nguồn gợi ý. Báo cáo, chặn. Tự ẩn khi vượt ngưỡng báo cáo.
Lộ bài FRIENDS hoặc PRIVATE qua gợi ý	Lọc quyền riêng tư ở tầng lọc và lọc lại trước khi trả. Test tự động cho từng mức quyền riêng tư.
Vòng phản hồi, video phổ biến càng phổ biến	Dành 10 đến 15% vị trí cho khám phá. Nhóm lưu lượng cho video mới. Theo dõi mức tập trung.
Bơm lượt xem giả	Giới hạn tần suất. Chỉ tính lượt xem vượt ngưỡng thời gian. Loại sự kiện bất thường.
Dịch vụ ML lỗi hoặc chậm	Thời hạn gọi ngắn và rơi về Phương án 1. Cache lô gợi ý gần nhất.
Giới hạn của transaction pooler Supabase	Truy vấn ngắn, ghi theo lô. Việc nặng (tổng hợp, huấn luyện) dùng kết nối trực tiếp.
Phá hợp đồng API hoặc chế độ mock	Chỉ thêm endpoint và migration cộng thêm. Dev B duyệt trước. Mọi service mobile giữ nhánh mock.

10. VIỆC CẦN QUYẾT ĐỊNH

1. Chọn phương án. Đề xuất: Phương án 7.
2. Chọn đường ống video cho bản deploy đầu tiên: V1, V2 hay V3.
3. Nơi chạy API, Redis và worker. Repo hiện chưa có cấu hình deploy.
4. Có chấp nhận thêm Python vào repo cho Phương án 5 hay không.
5. Chính sách cho bài FRIENDS trong Reels: loại hẳn hay chỉ hiện cho bạn bè.
6. Ai chịu trách nhiệm kiểm duyệt nội dung và xử lý báo cáo.

11. TÀI LIỆU THAM KHẢO

Danh mục lập theo trí nhớ của người soạn và chưa được tra cứu lại. Cần kiểm tra trước khi trích dẫn chính thức.

1. Covington, Adams, Sargin. Deep Neural Networks for YouTube Recommendations. RecSys 2016.
2. Yi và cộng sự. Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations. RecSys 2019.

3. Zhao và cộng sự. Recommending What Video to Watch Next: A Multitask Ranking System. RecSys 2019.
4. Ma và cộng sự. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts (MMoE). KDD 2018.
5. Tang và cộng sự. Progressive Layered Extraction (PLE). RecSys 2020.
6. Liu và cộng sự. Monolith: Real Time Recommendation System With Collisionless Embedding Table. ByteDance, 2022.
7. Naumov và cộng sự. Deep Learning Recommendation Model for Personalization and Recommendation Systems (DLRM). Facebook, 2019.
8. Zhan và cộng sự. Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation. KDD 2022.
9. Pal và cộng sự. PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest. KDD 2020.
10. Kang, McAuley. Self-Attentive Sequential Recommendation (SASRec). ICDM 2018.
11. Chen và cộng sự. Fast Greedy MAP Inference for Determinantal Point Process to Improve Recommendation Diversity. NeurIPS 2018.
12. Rajput và cộng sự. Recommender Systems with Generative Retrieval (TIGER). NeurIPS 2023.
13. Gao và cộng sự. KuaiRec: A Fully-observed Dataset and Insights for Evaluating Recommender Systems. CIKM 2022.
14. Ni và cộng sự. MicroLens: A Content-Driven Micro-Video Recommendation Dataset at Scale. 2023.
15. TikTok Newsroom. How TikTok recommends videos #ForYou. 2020.
16. Instagram Engineering. Scaling the Instagram Explore recommendations system. 2023.
17. Mã nguồn thuật toán gợi ý của Twitter: github.com/twitter/the-algorithm. 2023.
18. Gorse, engine gợi ý mã nguồn mở: github.com/gorse-io/gorse.